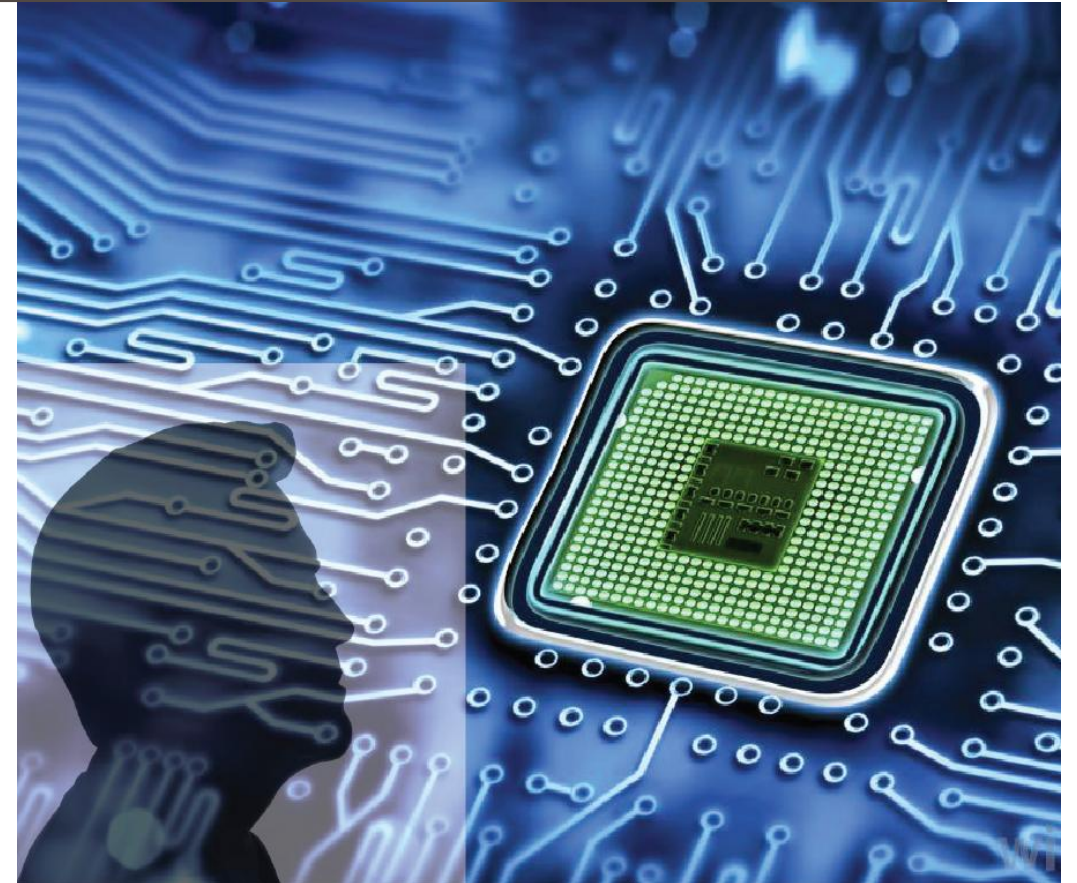


ADVANCED DIGITAL DESIGN AND VERIFICATION

Week-13 Lecture-1 UVM

Sneh Saurabh
18th November 2025



Universal Verification Methodology (UVM)

UVM Architecture: Environment (**uvm_env** class)

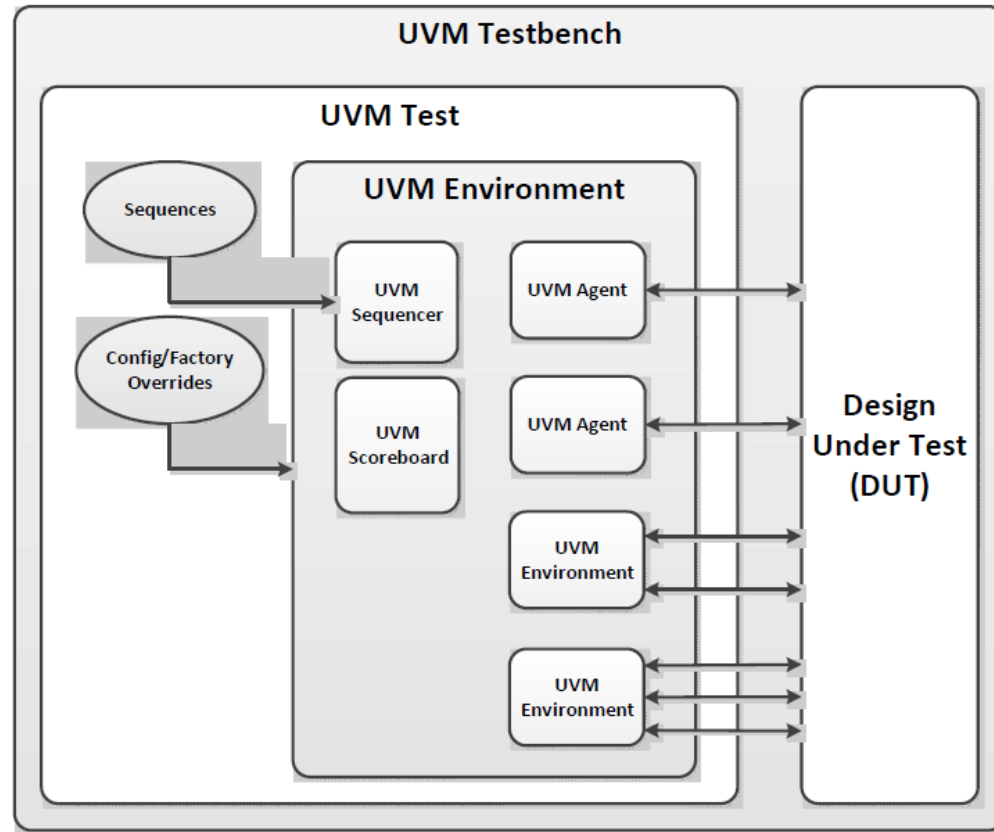


Figure 1—Typical UVM Testbench Architecture

UVM Environment

- Is a hierarchical component that groups together other verification components that are interrelated.
- Typically contains:
 - UVM Agents
 - UVM Scoreboards
 - Other UVM Environments

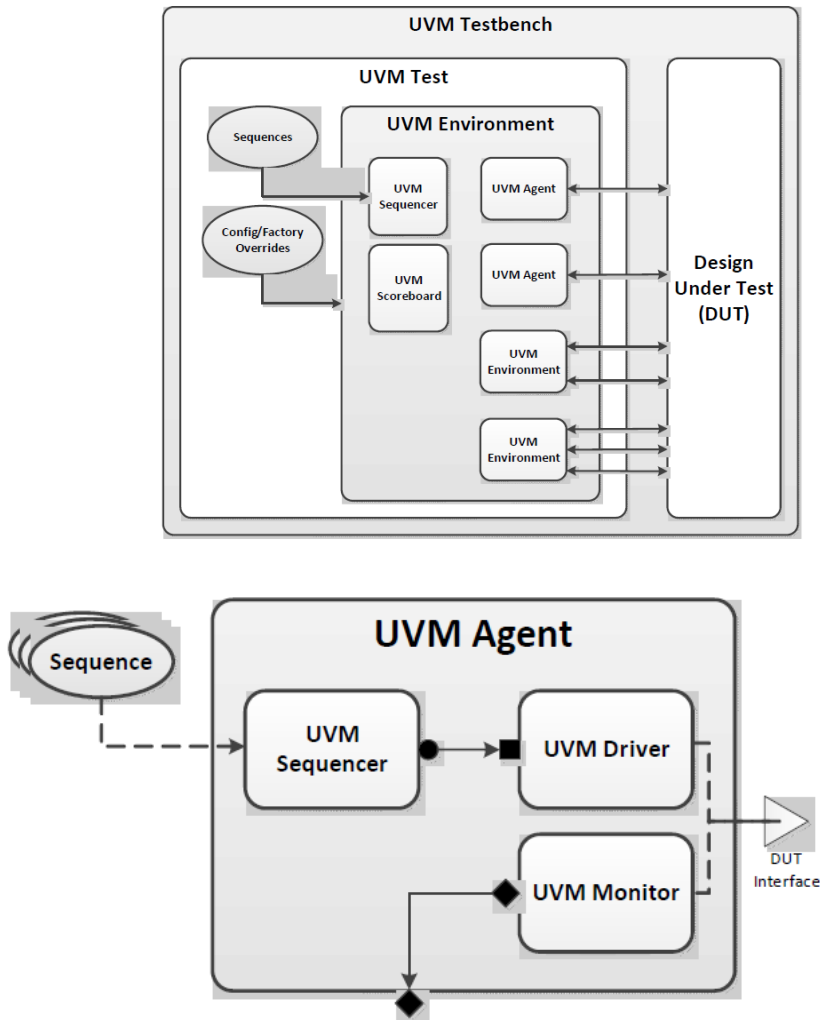
Example:

- UVM Environment per IP (e.g., PCIe, USB, Memory Controller, etc.)
- IP Environments are grouped into Cluster Environments (e.g., IO, Processor, etc.)
- Cluster Environment are grouped to top-level SoC Environment.

UVM: Transaction-level Modeling (TLM)

- Verification environment working at the higher level helps improve productivity and effectiveness
 - Example: verifying a DUT that handles packets and processes instructions, is easier than signal-level verification
 - Generating stimulus, collecting coverage data, and other verification task is more effective at the transaction level
 - Interface to the DUT ultimately is represented by signal-level activity
-
- UVM provides a set of transaction-level communication interfaces and channels that can connect components at the transaction level.
-
- TLM-1 (standardized in 2005) and TLM-2.0 (standardized in 2009) are widely used in industry
 - Both were built in SystemC [discussed earlier]
 - Subset of both these standards has been implemented in SystemVerilog and is available as part of UVM.

UVM Architecture: Agent (**uvm_agent** class)



UVM Agent

- A hierarchical component that groups together other verification components that are dealing with a specific DUT interface
- Typically contains:
 - UVM Sequencer to manage stimulus flow
 - UVM Driver to apply stimulus on the DUT interface
 - UVM Monitor to monitor the DUT interface
- UVM Agents might include other components, like coverage collectors, protocol checkers, a TLM model, etc.

Figure 2—UVM Agent

Sequencer, Driver, and Monitor (**uvm_sequencer**, **uvm_driver**, and **uvm_monitor** class)

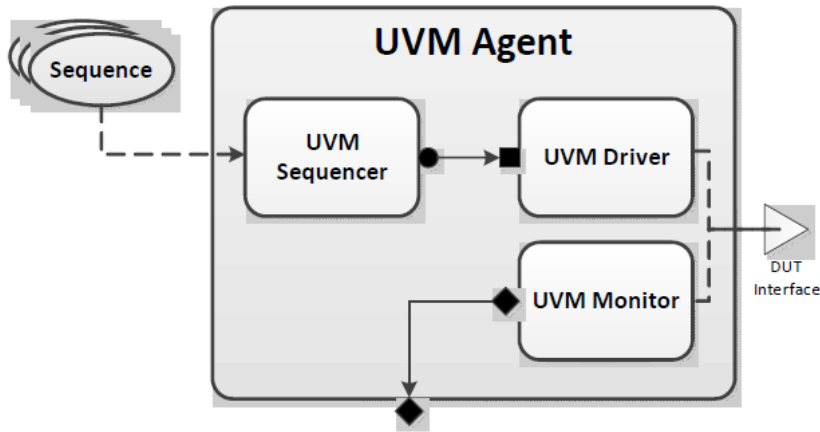


Figure 2—UVM Agent

UVM Sequence

- Is an object that contains a behavior for generating stimulus

UVM Sequencer

- An arbiter for controlling transaction flow from multiple stimulus sequences
 - Controls the flow of UVM Sequence Items transactions generated by one or more UVM Sequences.

UVM Driver

- Receives UVM Sequence Item (transactions) from the UVM Sequencer and applies (drives) it on the DUT Interface
- Converting transaction-level stimulus into pin-level stimulus
 - Has a TLM port to receive transactions from sequencer
 - Has access to the DUT interface to drive the signals.

UVM Monitor

- Samples the DUT interface (at pin-level) and sends transactions to the rest of the UVM Testbench for analysis
 - Has access to the DUT interface and TLM analysis port to broadcast information
 - Can do coverage collection, checking, logging, recording, etc

UVM Architecture: Scoreboard (**uvm_scoreboard** class)

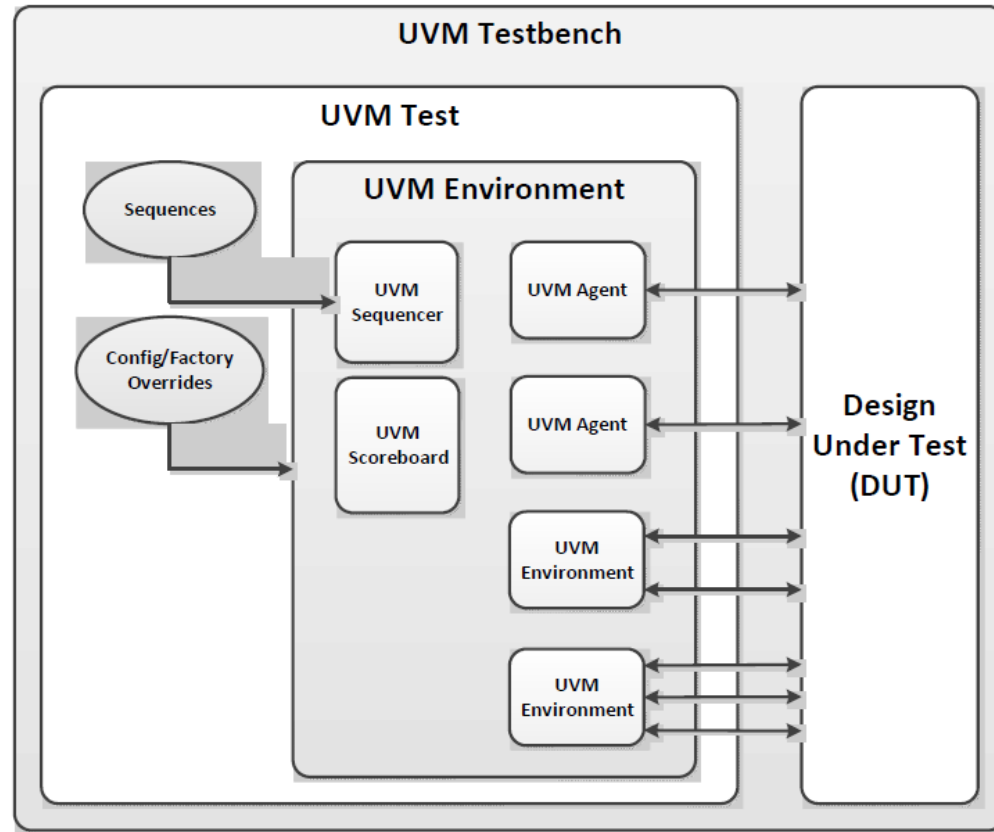


Figure 1—Typical UVM Testbench Architecture

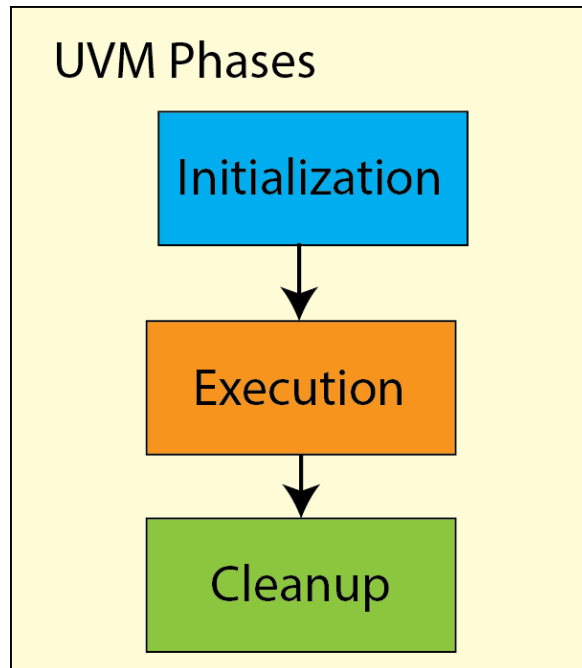
UVM Scoreboard

- Checks the behavior of the given DUT
 - Tasks
 - Receives transactions carrying inputs and outputs of the DUT through UVM Agent analysis ports
 - Runs the input transactions a reference model (also known as the predictor) to produce expected transactions
 - Compares the expected output versus the actual output
-
- Different methodologies on how to implement the scoreboard depending on:
 - Nature of the reference model
 - How to communicate between the scoreboard and the rest of the testbench

UVM: Phases

UVM: Phases

- UVM provides a standardized flow for testing divided into several phases
 - Allow systematic initialization, execution, and clean up for test benches



Major phases:

- Initialization:
 - **Build phase:** creates and configures testbench components (e.g., drivers, monitors, sequencers).
 - **Connect phase:** connects components using TLM or exports
- Execution: runs the actual test (stimulus generation, driving, and monitoring in the run phase)
- Cleanup:
 - **Extract phase:** gathers data (for coverage etc.) from monitor
 - **Check phase:** performs checks to validate functional correctness
 - **Report phase:** generates reports

UVM: Phase Example

```
`include "uvm_macros.svh"
import uvm_pkg::*;

module top;
    initial begin
        run_test(); // Start UVM test
    end
endmodule
```

```
[sneh@edatools-server1 uvm-2]$ ncverilog -uvm -uvmhome CDNS-1.1d
tb_top.sv +UVM_TESTNAME=my_test
```

```
UVM_INFO @ 0: reporter [RNTST] Running test my_test...
Building components...
Connecting components...
Running simulation...
Performing checks...
Cleaning up...
```

```
class my_test extends uvm_test;

    `uvm_component_utils(my_test)

    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        $display("Building components..."); // Instantiate sub-components
    endfunction

    function void connect_phase(uvm_phase phase);
        $display("Connecting components..."); // Connect TLM ports
    endfunction

    task run_phase(uvm_phase phase);
        $display("Running simulation..."); // Generate stimulus
    endtask

    function void check_phase(uvm_phase phase);
        $display("Performing checks...");
        // Validate results
    endfunction

    function void final_phase(uvm_phase phase);
        $display("Cleaning up...");
        // Final actions
    endfunction

endclass
```

UVM: Order of Phase Execution in Hierarchy

- Build phase: top-down
- Connect phase: bottom-up
- Run phase: parallel
- Extract phase: bottom-up
- Check phase: bottom-up
- Report phase: bottom-up
- Final phase: top-down

UVM: Top-down Build Phase (1)

```
1 `include "uvm_macros.svh"
2 import uvm_pkg::*;
3
4 class MyDriver extends uvm_driver;
5     `uvm_component_utils(MyDriver)
6
7     function new(string name, uvm_component parent);
8         super.new(name, parent);
9     endfunction
10
11     function void build_phase(uvm_phase phase);
12         super.build_phase(phase);
13         $display("Building MyDriver ...");
14     endfunction
15
16 endclass
```

```
18 class MyMonitor extends uvm_monitor;
19     `uvm_component_utils(MyMonitor)
20
21     function new(string name, uvm_component parent);
22         super.new(name, parent);
23     endfunction
24
25     function void build_phase(uvm_phase phase);
26         super.build_phase(phase);
27         $display("Building MyMonitor ...");
28     endfunction
29
30 endclass
```

UVM: Top-down Build Phase (2)

```
34 class MyEnv extends uvm_env;
35
36     MyDriver drv;
37     MyMonitor monitor;
38     `uvm_component_utils(MyEnv)
39
40     function new(string name, uvm_component parent);
41         super.new(name, parent);
42         drv = MyDriver::type_id::create("drv", this);
43         monitor = MyMonitor::type_id::create("monitor", this);
44     endfunction
45
46     function void build_phase(uvm_phase phase);
47         super.build_phase(phase);
48         $display("Building MyEnv ...");
49     endfunction
50
51 endclass
```

UVM: Top-down Build Phase (3)

```
54 class MyTest extends uvm_test;
55
56     MyEnv env;
57
58     `uvm_component_utils(MyTest)
59
60     function new(string name, uvm_component parent);
61         super.new(name, parent);
62         env = MyEnv::type_id::create("env", this);
63     endfunction
64
65     function void build_phase(uvm_phase phase);
66         $display("Building MyTest...");
67     endfunction
68
69 endclass
```

```
71 module top;
72     initial begin
73         run_test("MyTest"); // Start UVM test
74     end
75 _endmodule
```

```
UVM_INFO @ 0: reporter [RNTST] Running test MyTest...
Building MyTest...
Building MyEnv ...
Building MyDriver ...
Building MyMonitor ...
```

UVM: Bottom-up Connect Phase (1)

```
1 `include "uvm_macros.svh"
2 import uvm_pkg::*;
3
4 class MyDriver extends uvm_driver;
5     `uvm_component_utils(MyDriver)
6
7     function new(string name, uvm_component parent);
8         super.new(name, parent);
9     endfunction
10
11     function void connect_phase(uvm_phase phase);
12         $display("Connecting MyDriver ...");
13     endfunction
14
15 endclass
16
17 class MyMonitor extends uvm_monitor;
18     `uvm_component_utils(MyMonitor)
19
20     function new(string name, uvm_component parent);
21         super.new(name, parent);
22     endfunction
23
24     function void connect_phase(uvm_phase phase);
25         $display("Connecting MyMonitor ...");
26     endfunction
27
28 endclass
```

UVM: Bottom-up Connect Phase (2)

```
32 class MyEnv extends uvm_env;
33
34     MyDriver drv;
35     MyMonitor monitor;
36     `uvm_component_utils(MyEnv)
37
38     function new(string name, uvm_component parent);
39         super.new(name, parent);
40         drv = MyDriver::type_id::create("drv", this);
41         monitor = MyMonitor::type_id::create("monitor", this);
42     endfunction
43
44     function void connect_phase(uvm_phase phase);
45         $display("Connecting MyEnv ...");
46     endfunction
47
48 endclass
```


UVM: Bottom-up Connect Phase (3)

```
51 class MyTest extends uvm_test;
52
53     MyEnv env;
54
55     `uvm_component_utils(MyTest)
56
57     function new(string name, uvm_component parent);
58         super.new(name, parent);
59         env = MyEnv::type_id::create("env", this);
60     endfunction
61
62     function void connect_phase(uvm_phase phase);
63         $display("Connecting MyTest...");
64     endfunction
65
66 endclass
67
68 module top;
69     initial begin
70         run_test("MyTest"); // Start UVM test
71     end
72 endmodule
```

UVM_INFO @ 0: reporter [RNTST] Running test MyTest...
Connecting MyDriver ...
Connecting MyMonitor ...
Connecting MyEnv ...
Connecting MyTest...

UVM: Run Phase in Parallel (1)

```
1 `include "uvm_macros.svh"
2 import uvm_pkg::*;
3
4 class MyDriver extends uvm_driver;
5     `uvm_component_utils(MyDriver)
6
7     function new(string name, uvm_component parent);
8         super.new(name, parent);
9     endfunction
10
11 task run_phase(uvm_phase phase);
12     #5;
13     $display("Running MyDriver time=%0d...", $time);
14 endtask

```

```
--
18 class MyMonitor extends uvm_monitor;
19     `uvm_component_utils(MyMonitor)
20
21     function new(string name, uvm_component parent);
22         super.new(name, parent);
23     endfunction
24
25 task run_phase(uvm_phase phase);
26     #25;
27     $display("Running MyMonitor time=%0d...", $time);
28 endtask
29
30 endclass

```

UVM: Run Phase in Parallel (2)

```
34 class MyEnv extends uvm_env;
35
36     MyDriver drv;
37     MyMonitor monitor;
38     `uvm_component_utils(MyEnv)
39
40     function new(string name, uvm_component parent);
41         super.new(name, parent);
42         drv = MyDriver::type_id::create("drv", this);
43         monitor = MyMonitor::type_id::create("monitor", this);
44     endfunction
45
46     task run_phase(uvm_phase phase);
47         #10;
48         $display("Running MyEnv time=%0d...", $time);
49     endtask
```

UVM: Run Phase in Parallel (3)

```
54 class MyTest extends uvm_test;
55
56     MyEnv env;
57
58     `uvm_component_utils(MyTest)
59
60     function new(string name, uvm_component parent);
61         super.new(name, parent);
62         env = MyEnv::type_id::create("env", this);
63     endfunction
64
65     task run_phase(uvm_phase phase);
66         phase.raise_objection(this);
67         #50;
68         $display("Running MyTest time=%0d...", $time);
69         phase.drop_objection(this);
70     endtask
71
72 endclass
```

```
74 module top;
75     initial begin
76         run_test("MyTest");
77     end
78     initial begin
79         #120;
80     end
81 endmodule
```

UVM_INFO @ 0: reporter [RNTST] Running test MyTest...
Running MyDriver time=5...
Running MyEnv time=10...
Running MyMonitor time=25...
Running MyTest time=50...

UVM: Factory

UVM: Factory

- UVM Factory is a central mechanism to create and configure objects and components
- Makes testbenches more reusable, scalable, and modular.

- Allows:
 - Dynamic, creation of object: using their type or string name
 - Type overrides: substitutes one type of object or component with another

How to use factory?

- Components and objects must be registered using macros like:
 - `uvm_component_utils`
 - `uvm_object_utils`

```
class my_driver extends uvm_driver;
    `uvm_component_utils(my_driver)

    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction

endclass

my_driver driver = my_driver::type_id::create("driver", this);
```

Factories: Utilize Dynamic Creation of Object

```
1 `include "uvm_macros.svh"
2 import uvm_pkg::*;
3
4 class Env_Ver_1 extends uvm_env;
5     `uvm_component_utils(Env_Ver_1)
6
7     function new(string name, uvm_component parent);
8         super.new(name, parent);
9     endfunction
10
11     function void build_phase(uvm_phase phase);
12         super.build_phase(phase);
13         $display("Building environment using Env_Ver_1 ...");
14     endfunction
15
16 endclass
```

```
35 module top;
36     initial begin
37         run_test("my_test"); // Start UVM test
38     end
39 endmodule
```

```
18 class my_test extends uvm_test;
19
20     Env_Ver_1 env;
21
22     `uvm_component_utils(my_test)
23
24     function new(string name, uvm_component parent);
25         super.new(name, parent);
26     endfunction
27
28     function void build_phase(uvm_phase phase);
29         $display("Building components my_test...");
30         env = Env_Ver_1::type_id::create("Env_Ver_1", this);
31     endfunction
32
33 endclass
```

```
$ ncverilog -uvm tb_top_original.sv
```

```
UVM_INFO @ 0: reporter [RNTST] Running test my_test...
Building components my_test...
Building environment using Env_Ver_1 ...
```

Dynamic Creation of Object using Command line

```
18 class Env_Ver_1_Updated extends Env_Ver_1;
19     uvm_component_utils(Env_Ver_1_Updated)
20
21     function new(string name, uvm_component parent);
22         super.new(name, parent);
23     endfunction
24
25     function void build_phase(uvm_phase phase);
26         super.build_phase(phase);
27         $display("Adding updates to the environment ...");
28     endfunction
29
30 endclass
--
```

```
18 class my_test extends uvm_test;
19
20     Env_Ver_1 env;
21
22     `uvm_component_utils(my_test)
23
24     function new(string name, uvm_component parent);
25         super.new(name, parent);
26     endfunction
27
28     function void build_phase(uvm_phase phase);
29         $display("Building components my_test...");
30         env = Env_Ver_1::type_id::create("Env_Ver_1", this);
31     endfunction
32
33 endclass
```

```
ncverilog -uvm tb_top_new_env.sv +uvm_set_type_override=Env_Ver_1,Env_Ver_1_Updated
```

```
UVM_INFO @ 0: reporter [UVM_CMDLINE_PROC] Applying type override from the
command line: +uvm_set_type_override=Env_Ver_1,Env_Ver_1_Updated
Building components my_test...
Building environment using Env_Ver_1 ...
Adding updates to the environment ...
```


Dynamic Creation of Object using Override

```
18 class Env_Ver_1_Updated extends Env_Ver_1;
19     `uvm_component_utils(Env_Ver_1_Updated)
20
21     function new(string name, uvm_component parent);
22         super.new(name, parent);
23     endfunction
24
25     function void build_phase(uvm_phase phase);
26         super.build_phase(phase);
27         $display("Adding updates to the environment ...");
28     endfunction
29
30 endclass
--
```

```
$ ncverilog -uvm tb top override.sv
```

```
UVM_INFO @ 0: reporter [RNTST] Running test my_test...
Building components my_test...
Building environment using Env_Ver_1 ...
Adding updates to the environment ...
```

```
34 class my_test extends uvm_test;
35
36     Env_Ver_1 env;
37
38     `uvm_component_utils(my_test)
39
40     function new(string name, uvm_component parent);
41         super.new(name, parent);
42         set_type_override("Env_Ver_1", "Env_Ver_1_Updated");
43     endfunction
44
45     function void build_phase(uvm_phase phase);
46         $display("Building components my_test...");
47         env = Env_Ver_1::type_id::create("Env_Ver_1", this);
48     endfunction
49
50 endclass
--
```

UVM: Virtual Interface

Virtual Interface: Used Extensively in UVM

- A virtual interface in SystemVerilog is a data type that holds a reference to an actual interface instance.

- It allows classes in object-oriented testbenches to dynamically access signals and methods within an interface
- Bridges the gap between the dynamic class-based world and the static module-based world of Verilog/SystemVerilog.

Why do we use Virtual Interface in UVM?

- Decoupling: Testbench classes remain independent of specific module hierarchies.
- Reusability: The same driver/monitor classes can work with different instances of an interface.
- Dynamic Binding: The virtual interface can be reconfigured at runtime (e.g., for test-specific configurations).

Virtual Interface: Used Extensively in UVM

```
1 interface Bus_interface;
2   bit clk;
3   bit [7:0]data;
4 endinterface

6 class Driver;
7   virtual Bus_interface vbif;
8
9   task driveNext();
10    vbif.data = vbif.data + 1;
11  endtask
12 endclass

14 module top;
15   Bus_interface bif();
16   Driver drv = new();
17   initial begin
18     bif.clk = 0;
19     drv.vbif = bif;
20     forever #5 bif.clk = ~bif.clk;
21   end
22   always @(posedge bif.clk) begin
23     drv.driveNext();
24     $display("time: %0d data: %0d", $time, bif.data);
25   end
26   initial begin
27     #50;
28     $finish;
29   end
30 endmodule
```

```
ncsim> run
time: 5 data: 1
time: 15 data: 2
time: 25 data: 3
time: 35 data: 4
time: 45 data: 5
```

References

- **Universal Verification Methodology (UVM) 1.2 User's Guide**

https://www.accellera.org/images/downloads/standards/uvm/uvm_users_guide_1.2.pdf